SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS **COURSE CODE: CSA04**

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance

QUESTIONS: 5 **Total Marks: 100**

Annexure A

Debugging or Optimization task

Code of Conduct:

I M. Pranay (192525382)_ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M. Pranay

SAMPLE QUESTIONS

(QUESTIONS CAN BE CHANGED AS PER YOUR REQUIREMENTS)

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)

Process P2:
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)

Identify the exact condition that causes deadlock.
Suggest an optimisation to eliminate the deadlock without changing process logic.

(20 Marks) (BL4)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.
You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(20 Marks) (BL4)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB
- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(20 Marks) (BL4)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(20 Marks) (BL4)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.
Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(20 Marks)(BL4)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

TOTAL MARKS AWARDED **FACULTY SIGNATURE**

ANSWERS

1. Deadlock Debugging

Explanation

A deadlock occurs when two or more processes wait indefinitely for resources held by each other.

- Process P1 locks resource R1 and waits for R2.
- Process P2 locks resource R2 and waits for R1.
- Both processes wait for each other, so neither can continue.

Optimization: Make both processes acquire resources in the same order: R1 first, then R2. This prevents the circular-wait condition without changing the critical-section logic.

CODE:

```
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t R1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t R2 = PTHREAD_MUTEX_INITIALIZER;

void *process(void *arg)
{
   int id = *(int *)arg;

   pthread_mutex_lock(&R1);
   pthread_mutex_lock(&R2);

   printf("P%d: entered critical section\n", id);

   pthread_mutex_unlock(&R2);
   pthread_mutex_unlock(&R1);

   return NULL;
}
```

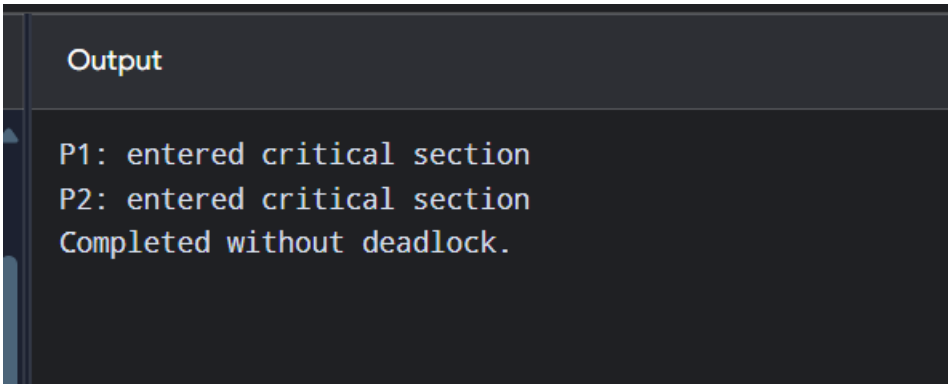
```
int main(void)
{
   pthread_t t1, t2;
   int p1 = 1, p2 = 2;

   pthread_create(&t1, NULL, process, &p1);
   pthread_create(&t2, NULL, process, &p2);

   pthread_join(t1, NULL);
   pthread_join(t2, NULL);

   printf("Completed without deadlock.\n");
   return 0;
}
```

OUTPUT:



Conclusion: Acquiring locks in a consistent order prevents the circular wait that causes this deadlock.

2. CPU Utilisation Problem

Explanation

The question describes CPU utilisation of only 30%, even though processes are ready to execute. The scheduler log shows processes frequently moving between the ready, waiting, and ready states.

This may indicate excessive I/O waiting or inefficient scheduling. However, the log alone does not prove the exact cause.

Optimization: Check I/O activity, reduce unnecessary blocking, and improve scheduling so that another ready process can use the CPU while a process waits for I/O.

CODE:

```
#include <stdio.h>

int main(void)
{
   int io_waiting = 1;
   int ready_process = 1;

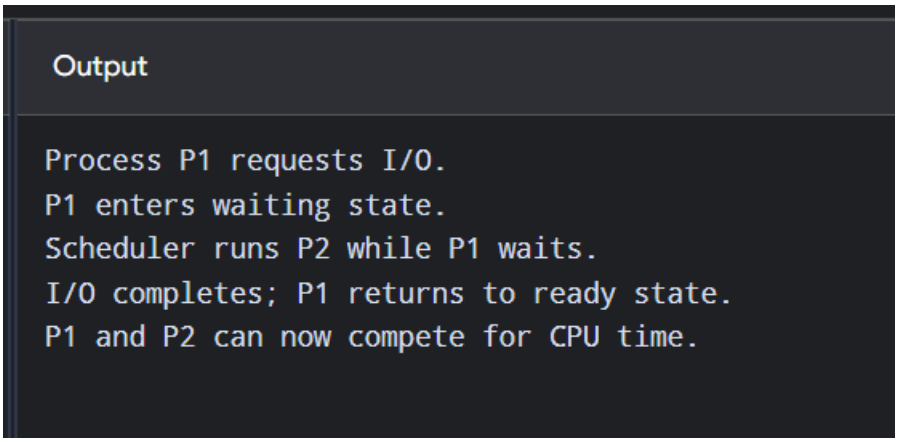
   printf("Process P1 requests I/O.\n");
   printf("P1 enters waiting state.\n");

   if (io_waiting && ready_process)
      printf("Scheduler runs P2 while P1 waits.\n");

   printf("I/O completes; P1 returns to ready state.\n");
   printf("P1 and P2 can now compete for CPU time.\n");

   return 0;
}
```

OUTPUT:



Conclusion: Scheduling another ready process during I/O waits can improve CPU utilisation. This program illustrates the concept; it does not measure actual CPU utilisation.

3. Memory Bottleneck

Explanation

The question provides the following memory details:

- Code memory = 5 MB
- Heap memory = 150 MB
- Stack memory = 1 MB per thread
- Number of threads = 25
- Available RAM = 2 GB (approximately 2048 MB)

Total listed memory:

5+150+(25*1) =180MB

The listed memory is less than available RAM, so these figures alone do not prove that insufficient RAM is causing the frequent page faults. Poor memory access patterns, memory pressure from other processes, or repeated allocation may also contribute.

Two optimizations:

1. Improve memory locality to reduce unnecessary page faults.
2. Reuse allocated memory and reduce unnecessary memory consumption.

CODE:

```
#include <stdio.h>

int main(void)
{
   int code = 5;
   int heap = 150;
   int stack_per_thread = 1;
   int threads = 25;
   int ram = 2048;

   int total_stack = stack_per_thread * threads;
   int total_memory = code + heap + total_stack;

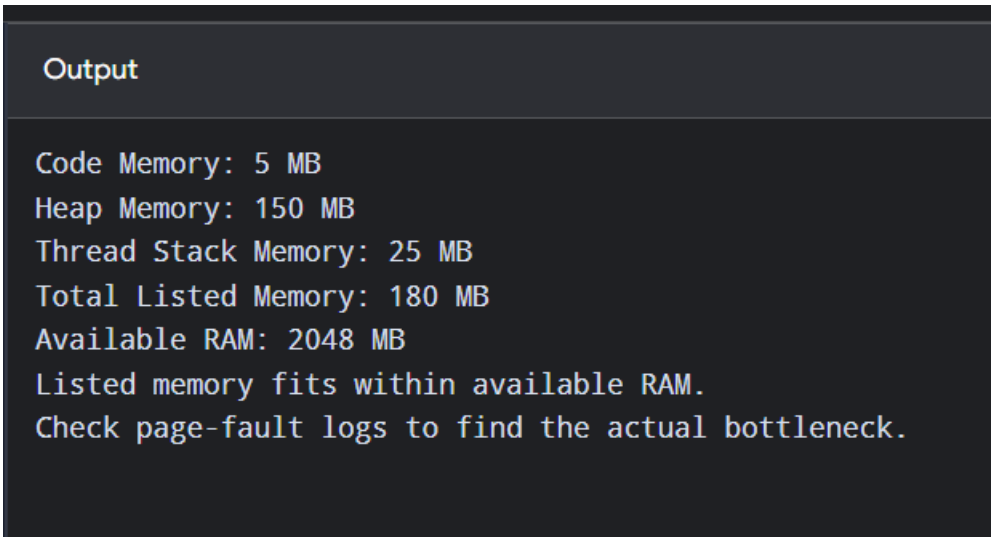
   printf("Code Memory: %d MB\n", code);
   printf("Heap Memory: %d MB\n", heap);
   printf("Thread Stack Memory: %d MB\n", total_stack);
   printf("Total Listed Memory: %d MB\n", total_memory);
   printf("Available RAM: %d MB\n", ram);

   if (total_memory <= ram)
      printf("Listed memory fits within available RAM.\n");
   else
      printf("Listed memory exceeds available RAM.\n");

   printf("Check page-fault logs to find the actual bottleneck.\n");

   return 0;
}
```

OUTPUT:



Conclusion: The calculated memory usage is 180 MB. Further investigation of page-fault logs and memory-access patterns is needed to identify the actual bottleneck.

4. Starvation in Multilevel Queue Scheduling

Explanation

Starvation occurs when a process waits indefinitely because other processes repeatedly receive CPU time before it.

In the question, the scheduler has three queues:

- Q1 – Interactive processes, highest priority, 5 ms quantum.
- Q2 – System processes, 10 ms quantum.
- Q3 – Batch processes, FCFS scheduling.

Process P5 belongs to Q3. If higher-priority queues continuously receive processes, P5 may never get CPU time.

Optimization: Apply aging. Aging gradually increases the priority of a process that has waited for a long time, giving P5 a better chance of execution.

CODE:

```
#include <stdio.h>

int main(void)
{
   int p5_priority = 3;
   int waiting_time = 0;
   int aging_limit = 3;

   printf("P5 starts in low-priority queue Q3.\n");

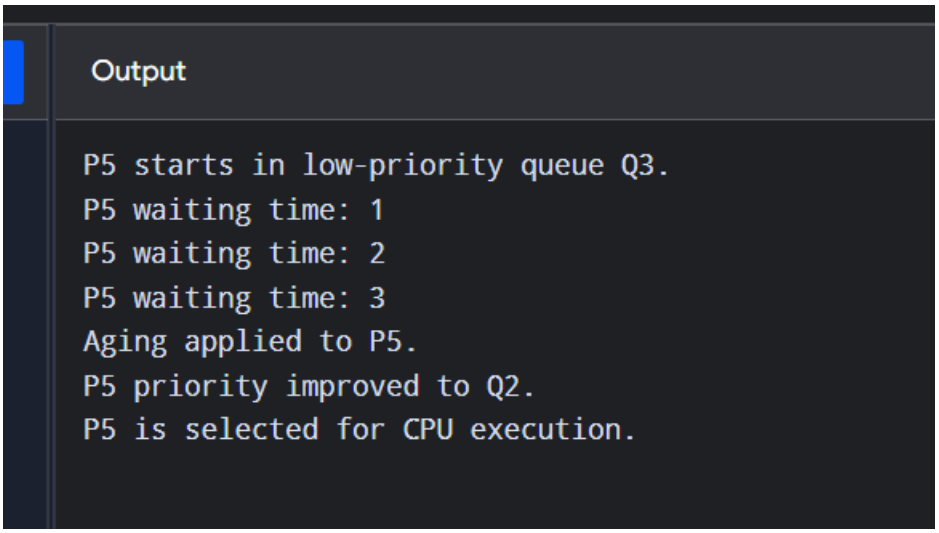
   while (waiting_time < aging_limit)
   {
      waiting_time++;
      printf("P5 waiting time: %d\n", waiting_time);
   }

   p5_priority = 2;

   printf("Aging applied to P5.\n");
   printf("P5 priority improved to Q%d.\n", p5_priority);
   printf("P5 is selected for CPU execution.\n");

   return 0;
}
```

OUTPUT:



Conclusion: Aging reduces starvation by increasing the priority of processes that have waited too long. This program demonstrates the idea; a real scheduler must implement the priority changes.

5. Context-Switch Overhead

Explanation

A context switch occurs when the operating system saves the state of one process or thread and restores the state of another.

The question gives these details:

- Context switches = 20,000 per second.
- Average CPU burst = 2 ms.
- Time quantum = 1 ms.
- High I/O waiting.

A short time quantum can cause frequent preemption, increasing context-switch overhead. I/O blocking and process wake-ups can also contribute to the number of switches.

Two optimizations:

1. Increase the time quantum appropriately, while considering response time.
2. Reduce unnecessary I/O operations and improve I/O handling.

CODE:

```
#include <stdio.h>

int main(void)
{
   int burst = 2;
   int quantum1 = 1;
   int quantum2 = 2;

   int preemptions1 = (burst - 1) / quantum1;
   int preemptions2 = (burst - 1) / quantum2;

   printf("CPU Burst: %d ms\n", burst);

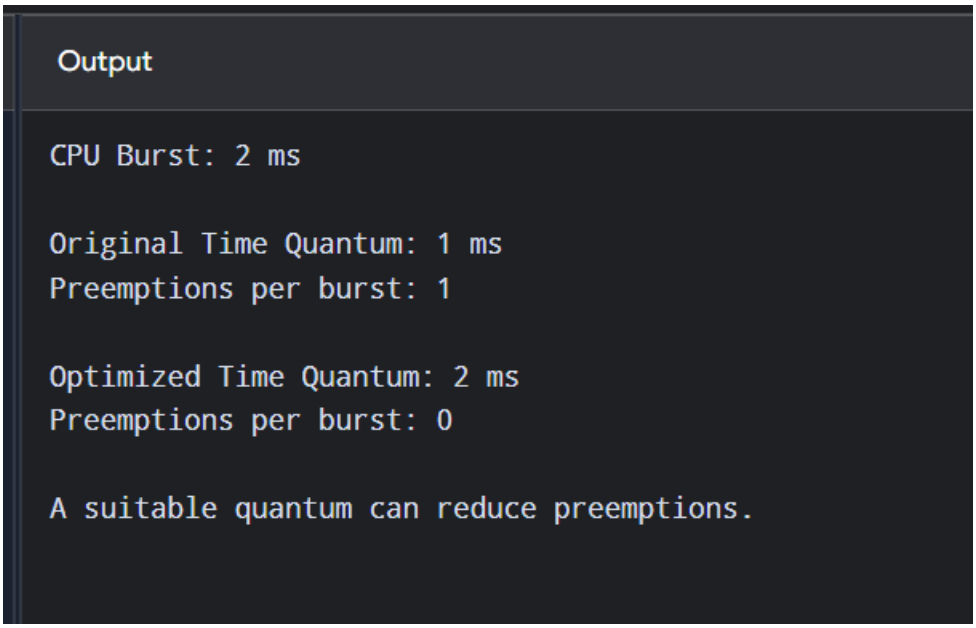
   printf("\nOriginal Time Quantum: %d ms\n", quantum1);
   printf("Preemptions per burst: %d\n", preemptions1);

   printf("\nOptimized Time Quantum: %d ms\n", quantum2);
   printf("Preemptions per burst: %d\n", preemptions2);

   printf("\nA suitable quantum can reduce preemptions.\n");

   return 0;
}
```

OUTPUT:



Conclusion: A suitable time quantum can reduce unnecessary preemptions and improve CPU efficiency. The code is a simplified calculation, not a real measurement of context switches per second.